

TP1 — Pipeline CI/CD multi-stages avec GitLab (version debutant)

Objectifs pedagogiques

A la fin de ce TP, vous saurez :

1. Creer un projet GitLab et y pousser du code
2. Ecrire un fichier `.gitlab-ci.yml` valide
3. Organiser un pipeline en **3 stages** (build, test, deploy)
4. Diviser votre configuration CI/CD en plusieurs fichiers avec `include`
5. Utiliser des **variables**, des **artifacts**, du **cache** et des **environments**
6. Declencher des jobs manuellement (deploiement en production)

Prerequis

- Un compte GitLab (gratuit sur <https://gitlab.com>)
- Git installe en local
- Node.js 18+ installe en local (pour tester avant de pousser)
- **Au moins 5 Go d'espace libre** sur votre disque
- Un editeur de texte (VS Code recommande)

****Pourquoi verifier l'espace disque ?****

Git a besoin d'ecrire des fichiers temporaires. Si le disque est plein, les commandes ``git commit``, ``git pull``, ``git merge`` vont echouer avec l'erreur ``Out of diskpace``.

Structure finale du projet

A la fin du TP, vous aurez cette arborescence :

```

TP1-CORRIGE/
├── .gitignore
├── .gitlab-ci.yml
├── .gitlab/
│   └── ci/
│       ├── build.yml
│       ├── test.yml
│       └── deploy.yml
├── package.json
├── src/
│   ├── index.js
│   └── utils.js
├── tests/
│   ├── api.test.js
│   └── utils.test.js

```

Tous ces fichiers sont déjà prêts dans ce dossier. Vous pouvez les utiliser comme référence, les copier dans un nouveau projet, ou suivre le guide étape par étape pour les recréer vous-même.

Etape 1 — Créer le projet sur GitLab

1. Allez sur <https://gitlab.com>
2. Cliquez sur **New project** > **Create blank project**
3. **Project name** : `tp1-pipeline-multistage`
4. **Visibility Level** : Private (ou Public si vous voulez partager)
5. **TRES IMPORTANT** : **Decochez** la case *"Initialize repository with a README"*
6. Cliquez sur **Create project**
7. GitLab vous affiche une page avec l'URL HTTPS du projet. **Gardez cette URL de côté.**

****Pourquoi décocher le README ?****

Si GitLab crée un commit initial (README.md), votre dépôt distant aura déjà un commit. Quand vous pousserez votre code local, Git refusera le push (car l'historique est divergent avec l'erreur `Updates were rejected because the tip of your current branch is behind`). En laissant le dépôt distant **complètement vide**, le premier push réussira sans conflit.

Etape 2 — Comprendre le fichier `package.json`

Regardez le fichier `package.json` du dossier. Il contient :

- **scripts** : les commandes raccourcies (`npm run build`, `npm test`, etc.) que la CI va exécuter
- **dependencies** : Express, le framework web de notre application
- **devDependencies** : Jest (tests) et Supertest (tests d'API)
- **jest.testEnvironment** : `"node"` : on teste du code Node.js, pas un navigateur

****Attention**** : on ne met ****PAS**** de `coverageThreshold` dans ce TP.

Si on exige 80% de couverture et que le code n'atteint pas ce seuil, les tests echouent automatiquement, ce qui bloque le pipeline. Pour un TP debutant, on garde la configuration simple.

Etape 3 — Comprendre le fichier `.gitignore`

Le fichier `.gitignore` indique a Git quels fichiers **ne jamais versionner**. Le plus important est `node_modules/`.

****Pourquoi c'est CRUCIAL ?****

Sans `.gitignore`, vous risquez de commiter le dossier `node_modules/` qui contient ****des centaines de megaoctets**** de dependances. Cela :

- Ralentit enormement `git push` et `git pull`
- Peut remplir votre disque dur
- Pollue l'historique Git
- N'a aucune utilite (les dependances sont reinstallees par `npm install` partout ou on en a besoin)

****Regle absolue en Node.js**** : `node_modules/` ne doit ****JAMAIS**** etre versionne.

Etape 4 — Comprendre le code source (`src/`)

`src/index.js`

Une API Express avec 3 routes :

- `GET /` : message de bienvenue
- `GET /health` : verification que l'API est en ligne
- `GET /add/:a/:b` : additionne deux nombres

****Pourquoi `module.exports = app` et la condition `require.main === module` ?****

Cela permet d'importer l'application dans les tests ****sans demarrer le serveur****. Le serveur ne démarre que si on execute `node src/index.js` directement. C'est la methode standard pour tester une API Express.

`src/utls.js`

Trois fonctions pures utilitaires (`add`, `multiply`, `isEven`) pour les tests unitaires.

****Pourquoi separer `index.js` et `utils.js` ?****

- `index.js` contient l'API Express, testee par les ****tests d'integration**** (qui simulent des requetes HTTP)
- `utils.js` contient des fonctions pures, testees par les ****tests unitaires**** (appels directs aux fonctions)
- Cette separation permet d'avoir 2 types de tests distincts qui s'executent en parallele dans la CI

Etape 5 — Comprendre les tests (tests/)

tests/utils.test.js

Tests unitaires rapides des fonctions de `utils.js`.

tests/api.test.js

Tests d'integration de l'API avec **Supertest**.

****Pourquoi `supertest` ?****

Supertest permet de tester une application Express ****sans la demarrer reellement****. Il simule des requetes HTTP directement sur l'objet `app`. C'est la methode standard et recommandee pour tester une API Node.js.

Etape 6 — Tester en local avant de pousser

Avant de configurer la CI, verifiez que tout fonctionne sur votre machine :

```
cd TP1-CORRIGE
npm install
npm test
```

****Pourquoi tester en local d'abord ?****

Si les tests echouent en local, ils echoueront aussi dans GitLab. Autant decouvrir les bugs maintenant plutot que de les voir dans le pipeline apres un push. Cela vous fait gagner du temps.

Vous devez voir quelque chose comme :

```
Tests:      8 passed, 8 total
```

Une fois que ca passe, vous pouvez **supprimer** `node_modules/` pour economiser de l'espace (il sera reinstaller par la CI) :

```
rm -rf node_modules
```

Etape 7 — Comprendre le fichier `.gitlab-ci.yml` (racine)

C'est le **point d'entree** du pipeline GitLab. Il contient :

```
variables:
  NODE_VERSION: "18"
  APP_NAME: "projet-demo-cicd"

stages:
  - build
  - test
  - deploy

include:
  - local: .gitlab/ci/build.yml
  - local: .gitlab/ci/test.yml
  - local: .gitlab/ci/deploy.yml
```

****Pourquoi cette structure ?****

- ****`variables`**** : centralise les valeurs reutilisables. Si on change la version de Node, on modifie un seul endroit.
- ****`stages`**** : l'ordre est important. ``build`` s'execute avant ``test``, qui s'execute avant ``deploy``.
- ****`include`**** : on pourrait tout mettre dans un seul fichier, mais avec 3 stages et plusieurs jobs, il deviendrait tres long. On separe pour la lisibilite.
- ****`workflow`**** : evite de declencher le pipeline sur chaque commit de chaque branche (ce qui gaspillerait des ressources).

Etape 8 — Comprendre `.gitlab/ci/build.yml`

Ce fichier definit le stage `build`. Il contient un **template reutilisable** `.base-node` et un **job** `build-app`.

****Pourquoi un template `.base-node`` ?****

Le point au debut (`.base-node``) signifie "job template". Il ne s'exécute pas tout seul, mais sert de modele pour d'autres jobs via ``extends``. Cela evite la duplication : les jobs ``build`` et ``test`` ont besoin des memes choses (npm install, cache), on les factorise.

****Pourquoi ``npm ci`` au lieu de ``npm install`` ?****

``npm ci`` est plus rapide et plus reproductible. Il installe ****exactement**** les versions du ``package-lock.json``, sans jamais le modifier. C'est la commande recommandee dans un contexte CI.

****Pourquoi les ``artifacts`` ?****

Le dossier ``dist/`` genere par le build est ****envoye aux jobs suivants**** (test-integration, deploy). Sans artifacts, les autres jobs ne verraient pas ce dossier. ``expire_in: 1 day`` supprime les artifacts apres 24h pour economiser l'espace GitLab.

Etape 9 — Comprendre `.gitlab/ci/test.yml`

Ce fichier definit le stage `test` avec **deux jobs en parallele** :

- `test-unitaire` : teste les fonctions de `utils.js`
- `test-integration` : teste l'API Express

****Pourquoi deux jobs de test ?****

Les deux jobs s'exécutent ****en parallele**** dans le stage ``test`` (GitLab lance tout ce qui peut tourner en meme temps pour aller plus vite). Le job ``test-integration`` a besoin des artifacts de ``build-app`` (d'ou le ``needs:``).

****Pourquoi ``policy: pull`` sur le cache ?****

Les jobs de test n'ont pas besoin de modifier le cache (pas de nouvelles dependances). ``pull`` signifie "lis le cache existant mais n'ecris rien dedans", ce qui accelere le job.

Etape 10 — Comprendre `.gitlab/ci/deploy.yml`

ATTENTION : c'est ici que se trouvent les pièges de syntaxe YAML les plus fréquents. Lisez attentivement la section suivante.

Ce fichier contient 3 jobs :

- **deploy-staging** : déploiement automatique sur staging après succès des tests
- **deploy-production** : déploiement **manuel** sur production (bouton play dans GitLab)
- **rollback-production** : retour arrière **manuel**

Les 3 pièges à éviter absolument

Piège 1 : le "deux-points + espace" dans les lignes script

YAML interprète : (deux-points suivi d'un espace) comme un séparateur cle/valeur. Si vous écrivez :

```
script:
  - echo "Application : mon-app"
```

GitLab renverra cette erreur :

```
jobs:deploy-staging:script config should be a string or a nested array of strings up to 10 levels deep
```

Solution : entourez la ligne de guillemets simples :

```
script:
  - 'echo "Application : mon-app"'
```

Piège 2 : **environment.action** accepte seulement 5 valeurs

Les seules valeurs valides pour **environment.action** sont :

- **start** (default)
- **stop**
- **prepare**
- **verify**
- **access**

rollback n'existe pas. Si vous écrivez **action: rollback**, le pipeline entier sera rejeté avec :

```
jobs:rollback-production:environment action should be start, stop, prepare, verify, or access
```

Solution : ne mettez simplement pas la cle **action** (elle vaut **start** par défaut).

Piège 3 : **jobs** dupliqués entre **.gitlab-ci.yml** et les **include**

Un job ne peut être défini **qu'une seule fois**. Si vous mettez **deploy-staging** dans **.gitlab-ci.yml** ET dans **.gitlab/ci/deploy.yml**, GitLab refusera le pipeline.

Règle : chaque job dans un seul fichier. **.gitlab-ci.yml** (racine) ne contient que les **variables**, **stages**, **include**, **workflow**.

Etape 11 — Pousser vers GitLab

Depuis le dossier **TP1-CORRIGE/**, exécutez :

```
# Initialiser un nouveau depot Git local
git init -b main

# Ajouter tous les fichiers (le .gitignore exclura automatiquement node_modules)
git add .

# Verifier ce qui va etre commite (doit afficher ~10 fichiers, PAS de node_modules)
git status

# Faire le premier commit
git commit -m "init: projet TP1 pipeline CI/CD multi-stages"

# Connecter au projet GitLab distant (REMPLACER L'URL par la votre)
git remote add origin https://gitlab.com/votre-user/tp1-pipeline-multistage.git

# Pousser le code vers GitLab
git push -u origin main
```

****Pourquoi `-b main`` ?****

Cree directement la branche `main`` au lieu de `master``. GitLab utilise `main`` comme branche par default depuis 2021, c'est la convention moderne.

****Pourquoi `-u origin main`` sur le push ?****

`-u`` (upstream) cree le lien entre votre branche locale et la branche distante. Les prochains `git push`` pourront etre faits sans arguments.

Etape 12 — Observer le pipeline sur GitLab

1. Ouvrez votre projet sur GitLab dans le navigateur
2. Dans le menu de gauche, cliquez sur **Build > Pipelines**
3. Vous devez voir un pipeline en cours d'execution avec votre commit
4. Cliquez sur le pipeline pour voir le detail

Ce que vous devez voir

- **Stage 1 — build** : 1 job (**build-app**) qui devient vert
- **Stage 2 — test** : 2 jobs en parallele (**test-unitaire**, **test-integration**) qui deviennent verts
- **Stage 3 — deploy** :
 - **deploy-staging** : **devient vert automatiquement** (declenche car on est sur **main**)
 - **deploy-production** : **reste en attente** avec un bouton play (declenchement manuel)
 - **rollback-production** : **reste en attente** avec un bouton play (declenchement manuel)

Tester le deployment manuel

1. Sur la page du pipeline, cliquez sur le bouton play a cote de **deploy-production**
2. Le job se lance et doit passer au vert
3. Vous venez de **deployer en production manuellement**

Recapitulatif des erreurs classiques

Erreur	Cause	Solution
Out of diskspace	Disque dur plein	Supprimer <code>node_modules/</code> , vider la corbeille, <code>docker system prune -a</code>
yaml invalid + script config error	: dans une ligne de script	Entourer la ligne de guillemets simples <code>'...'</code>
environment action should be start, stop...	action: rollback	Supprimer la ligne <code>action:</code>
Updates were rejected au git push	Le depot distant a des commits que vous n'avez pas	Ne jamais initialiser GitLab avec un README
npm ci can only install with existing package-lock	Pas de <code>package-lock.json</code>	Faire un <code>npm install</code> local une fois pour le generer
Jobs dupliques	Meme job dans <code>.gitlab-ci.yml</code> et dans un <code>include</code>	Un job ne peut etre defini qu'une seule fois
Tests echouent avec coverage threshold not met	Seuil de couverture trop strict dans <code>package.json</code>	Retirer la cle <code>coverageThreshold</code>

Valider sa syntaxe AVANT de pousser

Astuce pro : GitLab propose un **editeur de pipeline en ligne** qui verifie la syntaxe YAML avant que vous ne poussiez votre code.

1. Dans votre projet GitLab, allez dans **Build > Pipeline editor**
2. Onglet **Validate** : verifie la syntaxe YAML
3. Onglet **Visualize** : affiche le graphe des jobs
4. Onglet **Full configuration** : resout les `include` et affiche la configuration complete

Si vous doutez d'une modification, vous pouvez la tester dans cet editeur **sans commiter**.

Pour aller plus loin

Une fois ce TP reussi, vous pouvez explorer :

- **rules** avancees : declencher un job uniquement sur certaines branches ou tags

- **parallel** : executer un meme job N fois en parallele
- **services** : demarrer une base de donnees (PostgreSQL, Redis) pour les tests d'integration
- **trigger** : declencher des pipelines dans d'autres projets
- **GitLab Pages** : deployer automatiquement un site statique
- **Environments** : gerer plusieurs environnements avec historique de deploiements

Ressources

- Documentation officielle : <https://docs.gitlab.com/ee/ci/>
- Reference des mots-cles YAML : <https://docs.gitlab.com/ee/ci/yaml/>

Fin du TP1. Felicitations ! Vous avez mis en place un pipeline CI/CD multi-stages complet et fonctionnel.